```
Name:P.Srikar
Hall:2403a52002
```

```
# Import pandas for data handling and analysis
import pandas as pd

# Import numpy for numerical operations
import numpy as np

# Import torch for deep learning tensor operations
import torch

# Import seaborn and matplotlib for visualization
import seaborn as sns
import matplotlib.pyplot as plt

# Import Hugging Face dataset loader
from datasets import load_dataset

# Import BERT tokenizer and model for sequence classification
from transformers import BertTokenizer, BertForSequenceClassification

# Import training utilities from Hugging Face
from transformers import TrainingArguments, Trainer

# Import evaluation metrics from sklearn
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from sklearn.metrics import confusion_matrix, classification_report
```

```
# Load IMDb dataset from Hugging Face (contains train and test splits)
dataset = load_dataset("imdb")

# Display first 3 records from training dataset
print(dataset["train"][:3])

# Convert training dataset to pandas DataFrame for analysis
df = pd.DataFrame(dataset["train"])

# Print dataset shape (rows, columns)
print("Dataset Shape:", df.shape)

# Display class distribution (0 = negative, 1 = positive)
print(df["label"].value_counts())

# Plot class distribution to check balance
sns.countplot(x="label", data=df)

# Add title to plot
plt.title("Class Distribution (0 = Negative, 1 = Positive)")

# Show plot
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and resta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

README.md: 7.81k/? [00:00<00:00, 114kB/s]

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 29.9MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 33.4MB/s]

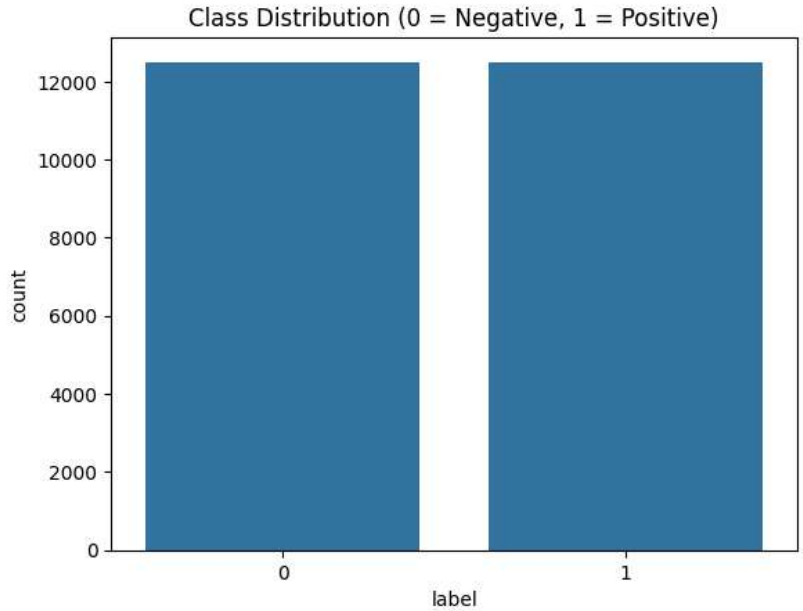
plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:00<00:00, 35.6MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 57386.78 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 66900.90 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 85408.64 examples/s]

```
{'text': ['I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first released in 1967. I also heard that at f
Dataset Shape: (25000, 2)
label
0    12500
1    12500
Name: count, dtype: int64
```




```
# Load pre-trained BERT tokenizer (uncased = lowercase text)
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

# Define tokenization function for dataset mapping
def tokenize_function(example):
    # Convert text into input_ids and attention_mask
```

```
    return tokenizer(
        example["text"],            # Input review text
        padding="max_length",       # Pad all sequences to same length
        truncation=True,            # Truncate long reviews
        max_length=256              # Set maximum sequence length
    )

# Apply tokenization to entire dataset (batched for speed)
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 1.09kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downl
vocab.txt: 100%  232k/232k [00:00<00:00, 3.12MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 9.63MB/s]
Map: 100%  25000/25000 [01:03<00:00, 413.93 examples/s]
Map: 100%  25000/25000 [00:46<00:00, 740.19 examples/s]
Map: 100%  50000/50000 [01:03<00:00, 644.67 examples/s]
```

```
# Rename label column to 'labels' for Trainer compatibility
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")

# Set dataset format to PyTorch tensors
tokenized_dataset.set_format(
    type="torch",                           # Convert to torch tensors
    columns=["input_ids", "attention_mask", "labels"]  # Required inputs for BERT
)

# Assign train dataset
train_dataset = tokenized_dataset["train"]

# Assign test dataset
test_dataset = tokenized_dataset["test"]
```

```
# Print number of training samples
print("Train Size:", len(train_dataset))

# Print number of testing samples
print("Test Size:", len(test_dataset))
```

```
Train Size: 25000
Test Size: 25000
```

```
# Load BERT model for sequence classification with 2 output labels
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",   # Pre-trained BERT base model
    num_labels=2           # Binary classification (negative, positive)
)
```

```
config.json: 100%                                    570/570 [00:00<00:00, 45.5kB/s]

model.safetensors: 100%                              440M/440M [00:08<00:00, 49.9MB/s]

Loading weights: 100%                                199/199 [00:00<00:00, 303.15it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.bias                        | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
classifier.bias                             | MISSING    |
classifier.weight                           | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```
# Define function to compute evaluation metrics
def compute_metrics(pred):
    # Extract true labels
    labels = pred.label_ids

    # Get predicted class by taking argmax of logits
    preds = pred.predictions.argmax(-1)

    # Compute precision, recall, and F1-score
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average="binary"
    )

    # Compute accuracy
    acc = accuracy_score(labels, preds)

    # Return metrics as dictionary
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

# Define training arguments
training_args = TrainingArguments(
    output_dir="./results",              # Directory to save model outputs
    learning_rate=2e-5,                  # Learning rate for optimizer
    per_device_train_batch_size=8,       # Batch size for training
    per_device_eval_batch_size=8,        # Batch size for evaluation
    num_train_epochs=2,                  # Number of training epochs
    # evaluation_strategy="epoch",         # Evaluate after each epoch
    save_strategy="epoch",               # Save model after each epoch
    logging_dir="./logs",                # Directory for logs
```

```
    # load_best_model_at_end=True          # Load best model based on evaluation
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
```

```
# Initialize Trainer with model, arguments, datasets, tokenizer, and metrics
trainer = Trainer(
    model=model,                        # BERT classification model
    args=training_args,                 # Training configuration
    train_dataset=train_dataset,        # Training data
    eval_dataset=test_dataset,          # Evaluation data
    compute_metrics=compute_metrics     # Evaluation metrics function
)

# Start model training
trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device p
  super().__init__(loader)
```

[57/6250 21:28 < 40:18:27, 0.04 it/s, Epoch 0.02/2]

Step	Training Loss

[62/6250 23:22 < 40:10:46, 0.04 it/s, Epoch 0.02/2]

Step	Training Loss

```
# Predict on test dataset
predictions = trainer.predict(test_dataset)

# Extract true labels
y_true = predictions.label_ids

# Extract predicted labels
y_pred = np.argmax(predictions.predictions, axis=1)

# Print classification report (precision, recall, F1, accuracy)
print(classification_report(y_true, y_pred))

# Compute confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Create confusion matrix plot
plt.figure(figsize=(6,5))

# Plot heatmap of confusion matrix
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Negative", "Positive"],
            yticklabels=["Negative", "Positive"])

# Label axes
plt.xlabel("Predicted")
plt.ylabel("Actual")

# Add title
```

```
plt.title("Confusion Matrix - BERT IMDb")

# Show plot
plt.show()
```